

Why the reset curriculum raises the ceiling and drops the floor on Franka Cabinet

Independent re-analysis of `logs/paper_logs` — 13 run groups × 16 seeds × 2500 iterations · 15 September 2026

Written without reference to the paper's own findings. Metric throughout: `dones/success_rate_margin`, the natural-episode mean of `clamp(drawer_pos/0.39, 0, 1)`, averaged over the last 100 iterations.

1 What the logs actually show

+0.07	2.0×	p = 0.28	4 / 16	0 / 16
median gain 0.732 → 0.805	seed-to-seed sd 0.113 → 0.228	Mann–Whitney $\delta = +0.23$	curriculum seeds below the baseline's <i>worst</i> seed	curriculum variants that remove the lower tail

The effect is not a widening of the baseline distribution. It is the appearance of a **new, disjoint failure cluster**. The baseline is itself bimodal — seven seeds land at 0.55–0.64, nine at 0.70–0.84 — but its support ends at 0.548. The curriculum moves twelve of sixteen seeds into the upper mode (a clear win) and pushes four seeds to 0.108, 0.367, 0.389 and 0.521, i.e. into a region the baseline never visits in 16 seeds. Mean performance is therefore flat (0.700 vs 0.702) while the median improves substantially.

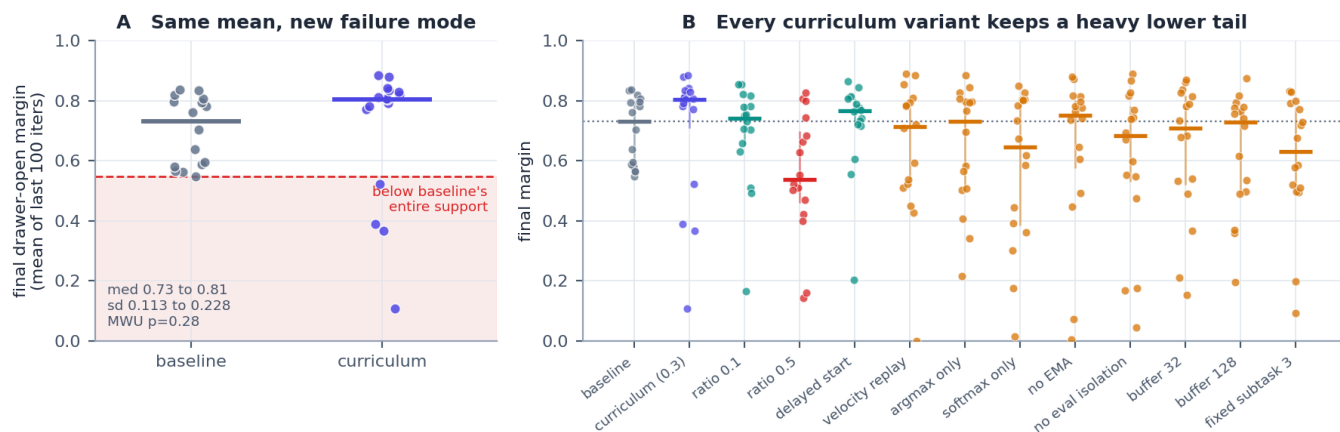


Figure 1. (A) Final performance, 16 seeds each. The curriculum's median rises; four seeds land below the baseline's entire empirical support. (B) All 13 run groups. Thick bar = median, whisker = IQR. Every curriculum variant — including the two that fix suspected bugs — retains a lower tail. Only *delayed start* visibly tightens the distribution.

The collapsed seeds are not late-training divergences. They are **stalls that are already visible by iteration 300** and never recover (Fig. 2B). The strongest early predictors of the final score, over the 16 curriculum seeds, are all measured in iterations 100–300: `subtasks/success_4` ($\rho = +0.77$, $p = 0.0006$), `curriculum/success_rate_2` ($\rho = +0.76$), and `curriculum/distribution_1` ($\rho = -0.57$, $p = 0.023$). The curriculum does not cause a late failure; it *amplifies whatever happened in the first 12% of training*, in both directions.

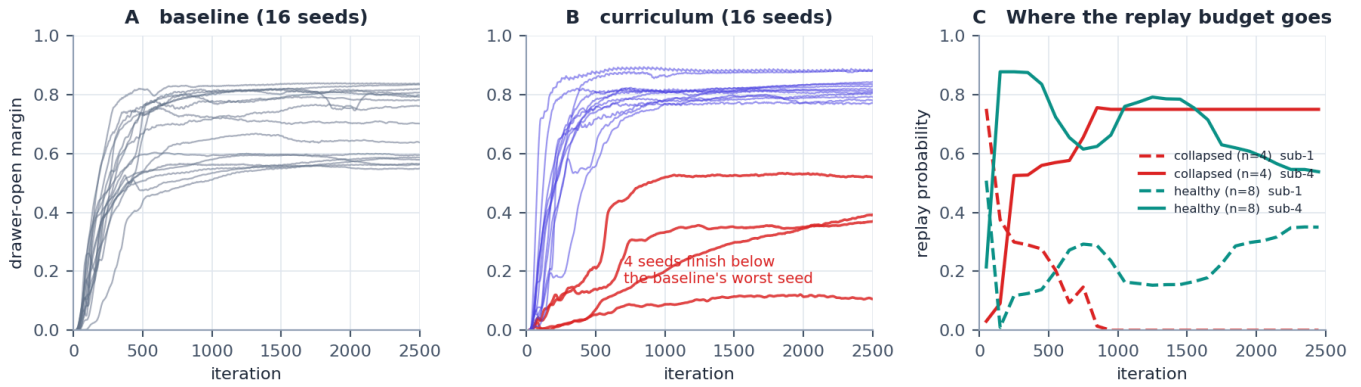


Figure 2. Per-seed learning curves (25-iteration moving average) and the scheduler's own output. (C) Mean replay probability assigned to subtask 1 (dashed) and subtask 4 (solid), grouped by outcome. Collapsed seeds spend their first ~200 iterations locked on subtask 1, then lock permanently on subtask 4. Healthy seeds escape subtask 1 within ~150 iterations and keep re-allocating mass back to it later.

2 Seven hypotheses

Ordered by my confidence that the mechanism is real *and* materially responsible for the lower tail. Each is stated as a claim, the mechanism, what in the existing logs supports or contradicts it, and the cheapest experiment that would settle it.

H1 · HIGHEST CONFIDENCE

Cabinet has no exploration bottleneck for the curriculum to buy, so only the costs remain

MECHANISM. The direct Franka Cabinet reward is dense and monotone in progress everywhere: $1.5 \cdot \text{dist_reward}$ (a smooth $1/(1+d^2)^2$ on end-effector-to-handle distance), $1.5 \cdot \text{rot_reward}$, $10 \cdot \text{drawer_pos}$, a finger-straddle shaping term, and step bonuses at 0.01 / 0.20 / 0.35 m. From the home pose, the reward gradient points at the solution. A reset-state curriculum buys one thing — access to states the policy cannot yet reach on its own — and Cabinet's shaping already grants that access. What is left is the bill: off-distribution starts, shortened episodes, and reset budget diverted away from the initial-state distribution.

EVIDENCE FOR. The cross-task pattern is the clean test and it is already in the logs (Fig. 3C). Factory, a genuine contact-rich insertion bottleneck with a razor-tight baseline (sd 0.009), improves with $\delta = +0.79$, $p < 0.001$. Lift, whose baseline collapses to 0 on 9 / 16 seeds, is rescued (14 / 16 seeds > 0.79 vs 6 / 16). Cabinet, whose baseline already solves the task on every seed, is the only one where the curriculum is statistically indistinguishable from baseline. The benefit tracks the size of the baseline's exploration gap; the harm does not.

EVIDENCE AGAINST. None found, but this is a between-task correlation over $n = 3$ tasks and cannot be causally separated from other differences (algorithm, horizon, reward scale).

TEST. Sparsify Cabinet's reward — drop dist_reward and rot_reward , keep only open_reward and the step bonuses — and re-run baseline vs curriculum, 8 seeds each. H1 predicts the baseline degrades sharply and the curriculum's advantage becomes significant. This is the single most informative experiment on this list: it converts a null result into a positive one by showing *when* the method is supposed to help.

H2 · HIGH CONFIDENCE

The replay budget is taken directly out of the distribution the metric is computed on

MECHANISM. 30% of resets do not start from the home pose, so the policy gets 30% less practice at the beginning of the task — which is exactly what the evaluation measures (time-averaged drawer openness from a natural start). Worse, replay episodes are much shorter, so their share of *resets* understates their share of the reset stream. Measured mean episode length: baseline 485.5 ± 6.8 steps, ratio 0.1 $\rightarrow 463.0 \pm 16.6$, ratio 0.3 $\rightarrow 411.5 \pm 57.7$, ratio 0.5 $\rightarrow 334.5 \pm 87.1$. Backing out the replay-episode length gives roughly 200 steps on average, and under 50 steps in several seeds — replay episodes reset 2–10× more often than natural ones and continuously re-draw from the same 64-slot buffer.

EVIDENCE FOR. A clean dose-response on the floor (Fig. 3B): baseline / 0.1 / 0.3 / 0.5 give 25th percentiles of 0.585 / 0.651 / 0.708 / 0.458 and minima of 0.548 / 0.164 / 0.108 / 0.142. Ratio 0.5 is the only variant *significantly worse*

than baseline ($p = 0.023$, $\delta = -0.48$). Separately, `env_compare/replay_task_success_gap_mean` is negative in every seed and every window: replayed episodes always look more competent than natural ones, by -0.09 in healthy seeds and -0.33 in collapsed ones, and this gap at iterations 300–700 is the second-best early predictor of the final score ($\rho = +0.74$, $p = 0.0009$). That is the signature of a policy that is good from replayed starts and not from the real one.

EVIDENCE AGAINST. The mean is non-monotone (0.3 is the best ratio), so a small dose is not purely harmful — consistent with a real but small benefit fighting a cost that grows with dose.

TEST. Hold the number of *natural* episodes fixed instead of the number of resets: add replay environments on top of the baseline's 16384 rather than converting 30% of them. If performance then matches or beats baseline at every ratio, the harm is budget theft, not the replay states themselves.

H3 · HIGH CONFIDENCE

The scheduler is a bang-bang controller, and its "soft" branch is uniform sampling

MECHANISM. Three compounding defects in `_update_distribution`:

- **The softmax does nothing.** `gaps` live in $[0, 1]$ and are then raised to `prob_exp = 2`, which pushes them *toward* zero, and a softmax over values spanning ~ 0.04 is flat to within 4%. The `prob_exp` "sharpening" parameter is a *flattening* parameter.
- **So the only real mechanism is the greedy branch.** `blend = clamp(margin / 0.10, 0, 1)` saturates at 1 whenever the top-two normalised confidence gap exceeds 0.10, which is most of the time (mean blend 0.79–1.00 across all windows). The effective controller is therefore *one-hot, or uniform, with nothing in between* — driven by a first difference of a noisy EMA.
- **Subtask 1 is structurally over-selected.** `gaps[0] = difficulty[0] - 0` compares subtask 1 against a phantom perfect predecessor. Combined with `success_rate = zeros` at init (`difficulty = [1,1,1,1]` → `gaps = [1,0,0,0]` → `margin = 1`), the curriculum *always* begins fully locked on subtask 1, and stays there until the EMA moves.

EVIDENCE FOR. Fig. 3A: under `ablation_soft` the distribution is $[0.233, 0.277, 0.224, 0.266]$ at iterations 1000–1200 and stays within 0.10 of uniform for the entire run — empirically confirming the softmax branch is uniform sampling. Fig. 2C: collapsed seeds hold 0.75 replay probability on subtask 1 through iteration ~ 200 ; the worst seed (final 0.108) holds 0.91–0.96 on subtask 1 for its first 500 iterations. Early `distribution_1` anti-correlates with the final score ($\rho = -0.57$, $p = 0.023$). And the blend beats both of its own branches (0.702 vs 0.646 argmax, 0.573 softmax), which is what you expect when one branch is degenerate and the other is uniform.

EVIDENCE AGAINST. Fixing the branches in isolation did not help: argmax-only and softmax-only are both worse than the blend. The defect is real, but on its own it is not the cause of collapse.

FIX AND TEST. Replace the difficulty heuristic with *learning progress* — $|\Delta \text{success rate}|$ over a sliding window, the standard Teacher-Student / ALP-GMM signal — normalise gaps to a simplex before the softmax, use an explicit temperature, put a floor of ~ 0.05 on every subtask so none can be starved, and define `gap1` against the natural-start success rate rather than against 0. Then re-run. The prediction is a much smoother `curriculum/selected` trace and a lower variance in outcomes.

H4 · HIGH CONFIDENCE — BEST SUPPORTED BY AN ABLATION YOU ALREADY RAN

The rule selects the subtask whose replay buffer is emptiest, which is a deadlock

MECHANISM. `success_buffer` is initialised to `zeros` and `world_ids = randint(0, success_buffer_size)` samples the whole buffer regardless of how much of it has been written. A subtask's buffer only fills once the policy reaches that subtask unaided. The difficulty rule, by construction, puts its mass on the subtask the policy reaches *least* — i.e. the one whose buffer is emptiest or stalest. Sampling an unwritten slot teleports the arm to the all-zeros joint vector, clamped to limits: for this Franka that is $\|q - q_{\text{default}}\| \approx 3.44$ rad, a configuration nothing in the task ever visits. So the budget is simultaneously (a) wasted on garbage states and (b) subtracted from the natural exploration that is the only thing that could fill the buffer. The loop is self-reinforcing in both directions, which is exactly the shape of a bimodal outcome.

EVIDENCE FOR. `ablation_delay` — which does nothing except withhold the curriculum until `progress > 0.15` (iteration 375) — is the best-behaved variant in the whole study: highest mean (0.720), *lowest* sd (0.159), tightest IQR (0.086 vs

0.122 curriculum and 0.214 baseline), and a collapse rate of 0.12 against 0.25 for both baseline and curriculum. It is the only variant that improves the floor. Its whole content is "do not use the buffer before it has content". Separately, the implied replay pose distance and variance at iterations 600–1000 anti-correlate with the final score ($\rho = -0.52$, $p = 0.039$ and $\rho = -0.58$, $p = 0.018$).

EVIDENCE AGAINST. The delay is confounded — it also skips the noisy early EMA and the subtask-1 lock-in of H3. Delay alone still leaves one seed at 0.203.

FIX AND TEST. **Buffer-readiness gating**, roughly ten lines: track per-subtask fill count, mask any subtask with fewer than m (say 16) distinct entries out of the sampling distribution, renormalise, and sample `world_ids` only from written slots. This subsumes the delay ablation and is strictly better-targeted. Run it at ratio 0.3 and at 0.5 — if H4 is right, the 0.5 run should no longer be worse than baseline.

H5 · MEDIUM CONFIDENCE

The buffer poisons itself: 16384 environments writing into 64 slots, biased toward the worst trajectories

MECHANISM. Three separate problems in the ring buffer, all invisible in the aggregate logs:

- **Single-instant collapse.** On any step where c environments newly cross a threshold, the writes go to $(\text{arange}(c) + \text{start}) \% 64$. With 16384 environments and a mass crossing, $c \gg 64$, so the call overwrites itself and the buffer ends up holding 64 states from *one simulation timestep* — near-identical poses, not a diverse replay memory. With ~4900 replay resets per rollout drawn from 64 slots, each state is duplicated ~77× in the batch.
- **Adverse selection.** Snapshots are only taken on the *first* crossing per episode, and a ring buffer keeps the most recent writes. Within an episode the last writers are the environments that crossed the threshold *latest*, i.e. the slowest, most marginal trajectories. The buffer preferentially retains the worst examples of each subtask.
- **Autophagy.** The buffer is filled from the policy's own rollouts and then fed back as initial states. A policy drifting toward extreme, limit-riding configurations snapshots those, replays them, and drifts further.

EVIDENCE FOR. Buffer capacity is irrelevant — 32 / 64 / 128 give 0.635 / 0.702 / 0.630 — which is what you expect if the binding constraint is *what gets written and how often it is wiped*, not how many slots exist. And the drift is visible: the implied replay-pose distance rises monotonically in collapsed seeds (2.97 → 4.33 rad across training) while healthy seeds peak at 3.96 and then *converge back down* to 3.56.

EVIDENCE AGAINST. The distance estimate is inferred from `curriculum/reset_distance` under an assumed 70/30 mixture, not measured directly. Direction of causation is not established — a failing policy produces extreme poses regardless.

FIX AND TEST. Replace the ring buffer with per-subtask *reservoir sampling* over the whole episode (so one mass crossing cannot wipe it), snapshot at a random delay after the crossing rather than at the crossing instant, and log buffer diversity directly — per-subtask pairwise distance and fill age. That last item is nearly free and would turn this hypothesis from inference into measurement.

H6 · MEDIUM CONFIDENCE — TWO CONCRETE BUGS

Replayed states are not physically valid, and one bug fires only on curriculum episodes

BUG A — DRAWER POSITION IS WRITTEN AS DRAWER VELOCITY. In `_reset_idx`:

```
self._cabinet.write_joint_state_to_sim(cabinet, cabinet, env_ids=env_ids)
```

The signature is `(position, velocity, ...)` and the same tensor is passed twice. For baseline resets `cabinet` is all zeros, so this is inert. For a replay it is not: a subtask-3 state starts the drawer at 0.20 m *moving outward at 0.20 m/s*, and a subtask-4 state at 0.38 m moving at 0.38 m/s. Termination fires at `drawer_pos > 0.39`. So a subtask-4 replay is launched 1 cm from a terminal state at 0.38 m/s, terminates within a few steps with no bootstrap, and the drawer velocity — which is an observation — is wildly off-distribution. **The severity of this bug is exactly proportional to how much mass the scheduler puts on subtasks 3 and 4, which is 0.61–0.78 for the entire second half of every run.**

BUG B — THE ACTION INTEGRATOR IS NEVER RESET. Actions are deltas: `targets = robot_dof_targets + speed_scale · dt · action · 7.5`, and `robot_dof_targets` is persistent per environment slot. `_reset_idx` calls `set_joint_position_target(robot_joint_pos, env_ids)`, but the next `_apply_action` unconditionally writes `self.robot_dof_targets` for *all* environments — so that reset write is dead code, and every new episode inherits the previous occupant's final commanded target as hidden state the policy cannot observe. The integrator moves at most 0.125 rad per step, so it takes ~10 steps to walk back; with stiffness 80 and effort limits of 87 / 12 N·m, a 1 rad target error means saturated torque for those steps. On a replay the arm is already interlocked with the handle, so this transient happens *in contact*. This is a genuine cross-episode MDP violation that affects baseline and curriculum differently.

BUG C — DOMAIN RANDOMISATION IS APPLIED IN JOINT SPACE WHILE IN CONTACT. `curriculum_dr = 0.02` rad on seven joints is up to ~2 cm of fingertip displacement, applied to a pose where the gripper straddles a thin handle. Penetration at reset produces depenetration impulses (`max_depenetration_velocity = 5.0`).

EVIDENCE FOR. Bugs A and B are verified by reading the source at the exact commit the runs were launched from (6e96761). The measured episode-length deficit per replay episode (~250–300 steps) is consistent with a large fraction of replays self-terminating.

EVIDENCE AGAINST — IMPORTANT. `ablation_velocity`, which fixes Bug A and adds velocity-aware replay, has the best maximum of any run in the study (0.889) but also a seed at 0.00004 and a *lower* mean (0.641). **Fixing the physics did not remove the tail.** But that ablation changed two things at once: it removed the spurious drawer velocity *and* began replaying stored robot and drawer velocities clamped at ± 2.0 , which injects its own instability. It is not a clean test.

TEST. Run the *minimal* fix — `cabinet_joint_vel = 0`, everything else identical to the main curriculum — as a one-line ablation. Also run Bug B's fix (`self.robot_dof_targets[env_ids] = robot_joint_pos` in `_reset_idx`) as a separate arm, on baseline as well as curriculum, since it changes the baseline too. These are the two cheapest experiments here and they clean up the story whatever the outcome.

H7 · MEDIUM CONFIDENCE — STRUCTURAL, NOT A BUG

The subtask grid is misspecified relative to the reward and the termination boundary

MECHANISM. The four subtasks are: end-effector within 0.20 m, within 0.10 m, drawer past 0.20 m, drawer past 0.38 m — with termination at 0.39 m. Two consequences:

- **The grid is badly spaced.** Subtask 3 → 4 is 18 cm of pulling on an already-grasped handle, essentially free. Subtask 2 → 3 contains the entire grasp-and-pull skill, which is where every collapsed seed is actually stuck (they reach `success_3` $\approx$ 0.61 but `success_4` $\approx$ 0.05). There is no rung on the ladder where the policy is failing.
- **Survivorship bias in the difficulty estimate.** Because success terminates the episode, an environment that solves subtask 4 is immediately removed from the population and contributes zero to `progression[:, 3, 0]` for the next several hundred steps. The difficulty estimator's denominator is polluted by exactly the successes it is trying to count, so `success_rate_4` can never approach 1 and the scheduler can never graduate from subtask 4. Empirically it never does: `distribution_4` sits at 0.61–0.78 at iteration 2500 in every single seed, healthy or collapsed.

There is also a reward-design consequence worth noting on its own: terminating at 0.39 m destroys the remaining $10 \times$ `drawer_pos` $\approx$ 3.9 per step the agent would have collected, so the return-maximising behaviour is to open the drawer to just under 0.39 and hold. The measured "success" metric rewards that, and subtask 4 sits 1 cm below that cliff — the scheduler spends most of its budget teleporting the policy into the one region where the value function is discontinuous.

EVIDENCE FOR. The distribution-4 lock is universal across seeds. The collapsed seeds' stall point (subtask 3 solved, subtask 4 not) lines up with the missing rung. Healthy seeds' `success_rate_4` saturates at 0.757, not near 1, as survivorship predicts.

EVIDENCE AGAINST. `ablation_default_3` (always replay subtask 3) is worse than the adaptive schedule (0.601 vs 0.702, $\delta = -0.31$), so a naively re-spaced fixed grid is not automatically better. The adaptive scheduler is adding value over a fixed one.

TEST. (i) Re-space: replace subtask 4's 0.38 m with 0.30 m and add a genuine intermediate rung between 2 and 3 ("handle grasped *and* drawer velocity > 0"). (ii) Independently, count subtask completions *per episode at termination* rather than as an instantaneous population mean, which removes the survivorship bias without touching anything else. (iii) As a diagnostic, run the curriculum with early termination disabled (time-out only) — if the tail shrinks, the value discontinuity is implicated.

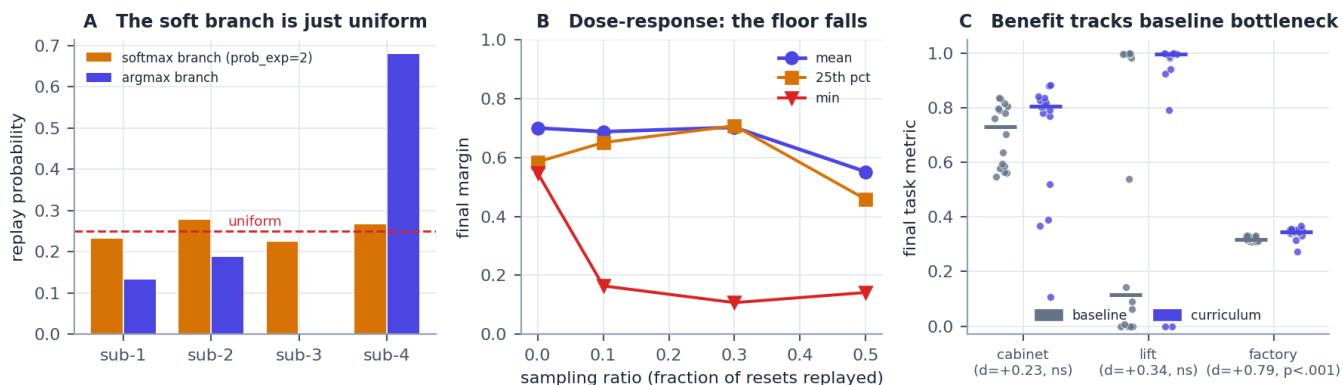


Figure 3. (A) The scheduler's softmax branch under `ablation_soft`, iterations 1000–1200: flat to within 0.05 of uniform, versus the argmax branch on the same statistic (H3). (B) Dose–response in the sampling ratio: the mean is roughly flat but the 25th percentile and the minimum fall away (H2). (C) Cabinet / Lift / Factory, baseline vs curriculum, all 16 seeds each; the curriculum's benefit scales with how badly the baseline is stuck (H1).

3 What I would run, in order

Ordered by information gained per GPU-hour. The first four are cheap and would change what the paper claims; the rest are follow-ups.

#	Experiment	Tests	Change	Prediction if right	Cost
1	Buffer-readiness gating	H4, H5	Mask subtasks with < 16 written buffer entries out of the sampling distribution; sample <code>world_ids</code> only from written slots.	Floor rises to $\geq$ baseline min; sd drops below 0.16; subsumes and beats <code>ablation_delay</code> .	16 seeds, ~15 h
2	Minimal velocity fix	H6-A	One line: pass a zero tensor as the cabinet velocity instead of reusing the position tensor.	Replay episode length rises toward 400+; the deepest collapses disappear; effect is larger at ratio 0.5.	16 seeds, ~15 h
3	Reset the action integrator	H6-B	<code>self.robot_dof_targets[env_ids] = robot_joint_pos in _reset_idx</code> . Run on <i>both</i> baseline and curriculum.	Baseline variance drops too; whatever remains of the curriculum gap becomes interpretable.	2×8 seeds, ~15 h
4	Held-out evaluation pass	all	Every 50 iterations, freeze the policy and roll out N envs from the home pose with the curriculum off; log that as the headline metric.	Removes the replay/eval confound from every figure in the paper. Essentially free.	~3% overhead
5	Sparsified Cabinet reward	H1	Drop <code>dist_reward</code> and <code>rot_reward</code> ; keep <code>open_reward</code> + bonuses.	Baseline degrades sharply; curriculum advantage becomes significant. Turns a null into a positive result.	2×8 seeds, ~15 h
6	Learning-progress scheduler	H3	$ \Delta\text{success} $ over a window; simplex-normalise before softmax; explicit temperature; 0.05 probability floor; <code>gap₁</code> vs natural-start rate.	Smoother <code>selected</code> trace; no subtask-1 lock-in; variance down.	16 seeds, ~15 h
7	Additive replay envs	H2	Add replay environments on top of 16384 instead of converting 30% of them.	Harm disappears at every ratio $\Rightarrow$ the cost was budget theft, not the states.	16 seeds, ~18 h
8	Re-spaced subtask grid	H7	Subtask 4 at 0.30 m; new rung "grasped and drawer moving" between 2 and 3; count completions at termination.	<code>distribution_4</code> stops saturating; the stuck seeds get a rung that matches where they fail.	16 seeds, ~15 h
9	Reservoir buffer + diversity logging	H5	Per-subtask reservoir sampling across the episode; snapshot at a random delay after crossing; log pairwise distance and fill age.	Converts H5 from inference to measurement; replay distance stops drifting outward in bad seeds.	16 seeds, ~15 h
10	Early-warning controller	H4, H2	Your <code>_run_curriculum_controller</code> is present but disabled. Trigger it on <code>replay_task_success_gap_mean</code> ($\rho = +0.74$) or <code>distribution_1</code> ($\rho = -0.57$) at iterations 100–300 rather than a reward slope.	Curriculum disables itself in the seeds it would have ruined; recovers baseline's floor while keeping the median.	16 seeds, ~15 h

A framing suggestion. The Cabinet result is currently a null with extra variance, which is hard to write up. The analysis above suggests it is better framed as a *scope* result: the method buys exploration, Cabinet's shaping already provides it, and the cost of buying something you already have is a heavier lower tail. Experiments 5 and 1 together would turn Cabinet from a weak point into the paper's clearest statement of when the method applies — which is a stronger contribution than a third task that mildly works.

4 Questions for you

These would change how I prioritise the list above; I have written the brief so it stands without them.

1. In earlier ablations, did the collapsed Cabinet seeds ever *recover* if trained past 2500 iterations, or is 2500 already past convergence? Every collapsed curve here is flat from ~1000 onward, which argues for a genuine attractor rather than slow learning — but you may have longer runs.
2. Was `controller_enabled` ever set to True in a run that is not in `paper_logs`? The slope-based trigger is implemented but disabled in all 13 groups, and if you tried it and it did not work, that constrains experiment 10 substantially.

3. Were the 16 Cabinet seeds 42–57, matching Factory? If the seeds are paired across baseline and curriculum, I can run a paired test instead of Mann–Whitney, which given the bimodality would be noticeably more powerful.
4. Do you already know whether the collapsed seeds fail by never grasping the handle, or by grasping and not pulling far enough? The logs say `success_3` ≈ 0.61 with `success_4` ≈ 0.05 , which reads as "pulls to ~ 20 cm and stops", but a rollout video from `2026-08-04_21-54-18/model_2499.pt` would settle H7's re-spacing question immediately.
5. Is there a reason the Lift and Cabinet ports use different snapshot contents (Lift stores object pose; Cabinet stores joint positions only, no velocities)? If the Lift version already stores velocities, that is a natural control for H6.

Sources: `logs/paper_logs/*` (TensorBoard scalars, 13 groups $\times$ 16 seeds); environment source read at commit `6e96761`

(`isaacsim_tasks/direct/franka_cabinet/franka_cabinet_env.py`) plus each group's stored `git/IsaacLab.diff`. Statistics: Mann–Whitney U (two-sided) and Cliff's delta on per-seed finals, $n = 16$ per group; Spearman ρ for within-group predictors, $n = 16$.